Ready

Great — before we start, I need the fictional interview context.

Please share:

- The company type, stage, and size.
- The product, platform, or business they operate.
- The role you're interviewing for.
- Any important domain, market, customer, scale, team, or organizational context.

Then I'll ask for the interview block specification and your prep artifacts.

IG Group

They are a FTSE100 fintech.
They offer a trading plataforma on stocks, futures and cfd to 1.3 million customers worldwide.
They are looking for strong lead engineer in Java.

Got it — I'm set with the company context: IG Group is a public fintech/trading platform, and the role is a hands-on **Java Lead Engineer** focused on trading systems and delivery.[1] [2]

Now send me the interview block specification, for example:

- leadership / behavioral
- Java technical deep dive
- system design
- culture / values
- mixed loop

After that, share the preparation artifacts you want me to use, like notes, stories, architecture summaries, or metrics.

Block 1: Mechanical Sympathy & High-Performance Java

Goal: Prove you can write code that doesn't just "work" but thrives under the pressure of millions of price updates.

The Concept: At IG's scale, standard synchronized blocks create bottlenecks. You need to understand how to share data between threads without "locking" the CPU.
Keyword Bank: CAS (Compare and Swap), AtomicReference, False Sharing, LMAX Disruptor, Memory Barriers, Thread Affinitiy.
Actionable Task: Research the LMAX Disruptor architecture. It is the industry standard for financial exchange "matching engines." Focus on how it uses a Ring Buffer to avoid Garbage Collection.

Good — I now have the interview block. This block is centered on low-latency Java, especially how the LMAX Disruptor uses a pre-allocated ring buffer, sequencing, CAS-based coordination, and memory-barrier-driven visibility to reduce contention and GC pressure in high-throughput systems.[9] [10] [11]

Now send me your fictional preparation artifacts, such as:

- Notes or study outline for Mechanical Sympathy / High-Performance Java.
- Your prepared explanation of CAS, false sharing, memory barriers, and thread affinity.
- Any system design notes for trading or price-streaming systems.
- Any past-project stories, benchmarks, incidents, or tuning examples you want me to use in answers.

Once I have that, I'll confirm I'm ready for the first interview question.

⁂

Perfect. That's enough context.

You're in the room. I'm the hiring manager — Principal Engineer at IG, responsible for the pricing and execution platform. We process millions of price ticks per second across thousands of instruments.

Let's go.

You've just joined the team. One of your first tasks is to review a shared price cache — a structure that holds the latest bid/ask for every instrument, updated by a market data thread and read by dozens of execution threads simultaneously.

A junior engineer wrote this:

```
public class PriceCache {
    private final Map<String, Price> cache = new HashMap<>();
```

```
    public synchronized void update(String instrument, Price price) {
        cache.put(instrument, price);
    }

    public synchronized Price get(String instrument) {
        return cache.get(instrument);
    }
}
```

What's wrong with this, and how do you fix it?

What's wrong is that the code makes every read and write contend on one monitor, so dozens of execution threads are serialized behind a single lock; that is safe, but it is exactly the kind of bottleneck low-latency systems try to remove. In a pricing path, that adds lock contention, context-switch cost, and tail-latency spikes, and it still leaves another risk: if `Price` is mutable, readers can observe torn or inconsistent state unless publication is designed carefully.[24] [25]

My first fix is `ConcurrentHashMap<String, Price>` plus an immutable `Price`, so reads are non-blocking and updates replace the whole value atomically. If the instrument universe is known and fixed, I'd go further: map instruments to integer ids and store the latest price in a preallocated array or `AtomicReferenceArray`, which removes hashing and avoids coarse locking on the hot path. For the absolute hottest path, I'd consider single-writer publication patterns inspired by the Disruptor: preallocate, publish by sequence, and design for cache-friendly reads.[26] [25] [24]

⁂

Good. You've correctly identified the core problem. Now let's go deeper.

You mentioned replacing the lock with `ConcurrentHashMap`. That buys you per-segment or per-bucket striping, but in a pricing system at IG's scale — thousands of instruments, millions of updates per second — is `ConcurrentHashMap` actually good enough?

What are its remaining limitations in this context, and what would you reach for instead?

I wouldn't stop at `ConcurrentHashMap`. It removes the single monitor, but it still pays for hashing, pointer chasing, volatile/CAS-style coordination, and extra memory overhead, which hurts cache locality and makes tail latency less predictable on a hot pricing path.[39] [40] [41]

With `String` keys and object-per-update `Price` instances, I'm also creating avoidable GC pressure and cache misses, which is exactly what low-latency designs try to minimize through preallocation and cache-friendly access patterns.[42] [43]

For this workload, I'd switch to a single-writer design: map each instrument to an integer id, keep a preallocated array or padded slot per instrument, and publish updates with sequence-based visibility so readers do lock-free reads.[43] [42]

If I need fan-out between threads, I'd use a Disruptor-style ring buffer for publication, but keep the latest-price view as an indexed, preallocated structure rather than a general-purpose map.[42] [43]

⁂

Good direction. You're thinking at the right level. But you've said "padded slot per instrument" — I'm going to hold you there.

Explain exactly what padding achieves in this context. What problem does it solve, what does the fix look like in Java, and what are the risks if you get it wrong?

Padding is there to prevent **false sharing**: two independent hot fields end up on the same CPU cache line, so when one thread updates its field, other cores keep invalidating and reloading that same line even though they are touching different logical data. In a price cache, that means thread A updating instrument 101 can slow down thread B reading or updating instrument 102 if their slots sit on the same line.[56] [57] [58]

In Java, the fix is to isolate hot mutable fields so each one effectively occupies its own cache line, either with `@Contended` or with explicit padded holder objects rather than a dense primitive array. Conceptually, I'd store each instrument in something like a `PaddedPriceSlot` with the published fields separated from other hot fields, and I'd prefer one writer per slot or per partition to keep ownership clear.[59] [60] [56]

If you get it wrong, you can have the worst kind of failure: the code is functionally correct but latency becomes noisy and throughput collapses under load. The tricky part is that manual padding in Java is not always reliable because object layout and field ordering can vary, so I'd validate with JOL or benchmarks rather than assume the padding worked just because I wrote extra longs.[60] [61] [62] [56]

⁂

Solid. You know what false sharing is and why padding matters. Now I'm going to make this concrete.

You said `@Contended` or explicit padding. Here's the situation:

You have 10,000 instruments. Each slot holds a bid, an ask, and a sequence number. You want to eliminate false sharing between slots.

Write me the padded slot class. Show me the actual Java. Then tell me what `@Contended` requires at the JVM level to actually work, and why you might choose explicit padding over it in a production deployment.

I'd write it as an explicitly padded holder so the hot mutable fields for one instrument are isolated from the next slot, and I'd keep the fields `volatile` so the market data writer publishes a coherent latest value to readers:

```
public final class PaddedPriceSlot {
    // left pad
    long p01, p02, p03, p04, p05, p06, p07;

    volatile long seq;
    volatile double bid;
    volatile double ask;

    // right pad
    long p11, p12, p13, p14, p15, p16, p17;
}
```

In practice I'd allocate one `PaddedPriceSlot` per instrument in an indexed array, ideally with one writer per slot or partition, and I'd benchmark the actual layout because HotSpot layout details matter.[71] [72]

`@Contended` can also do this, but for user code it only takes effect when the JVM is started with `-XX:-RestrictContended`; on newer JDKs you also need access to `jdk.internal.vm.annotation.Contended`, typically via `--add-exports java.base/jdk.internal.vm.annotation=ALL-UNNAMED` or the relevant module export. The padding width is JVM-controlled, and the default padding width is typically 128 bytes.[73] [74] [75] [76] [71]

I might still choose explicit padding in production because it is operationally simpler: no dependency on special JVM flags, no reliance on internal JDK annotations, and fewer surprises across environments or container launch configs. The downside is that manual padding is more fragile, so I'd only keep it if profiling proves it helps and layout checks confirm it actually separates hot fields the way I expect.[74] [76] [72] [71]

⁂

Good. The structure is right. Now I'm going to stress-test your understanding of what you just wrote.

Three sharp questions — take them one at a time.

First one:

You've marked `bid` and `ask` as `volatile double`. In the JVM, a `double` is 64 bits. On a 32-bit JVM, reads and writes to a `double` are not guaranteed to be atomic. But you're on a modern 64-bit JVM.

Is `volatile double` sufficient here to guarantee that a reader never sees a torn price — a bid or ask that is half-written? Justify your answer precisely, and tell me if there's a remaining race condition even if the write is atomic.

Yes — `volatile double` is sufficient to prevent a **torn** read or write of that individual `double`. The Java Language Specification states that reads and writes of `volatile long` and `volatile double` are always atomic, so a reader will not observe half the bits from one write and half from another.[87] [88]

But that only protects each field independently. It does **not** make the pair `(bid, ask)` atomic as one logical snapshot, and it does not guarantee the reader sees `bid`, `ask`, and `seq` from the same market update. A reader could legally observe the new `bid` and the old `ask`, or a new price with an old sequence, depending on read timing.[89] [90]

So the remaining race is a consistency race, not a torn-write race. In a production price slot, I'd use the sequence as the publication guard: writer updates bid/ask, then publishes `seq`; reader reads `seq`, then bid/ask, then re-reads `seq` and only accepts the snapshot if the sequence is unchanged and valid.[91] [87]

⁂

Good. The structure is right. Now I'm going to stress-test your understanding of what you just wrote.

Three sharp questions — take them one at a time.

First one:

You've marked `bid` and `ask` as `volatile double`. In the JVM, a `double` is 64 bits. On a 32-bit JVM, reads and writes to a `double` are not guaranteed to be atomic. But you're on a modern 64-bit JVM.

Is `volatile double` sufficient here to guarantee that a reader never sees a torn price — a bid or ask that is half-written? Justify your answer precisely, and tell me if there's a remaining race condition even if the write is atomic.

Yes. `volatile double` is enough to guarantee that an individual `bid` or `ask` read is not torn, because the JLS explicitly says reads and writes of `volatile long` and `volatile double` are always atomic.[103] [104]

But that does **not** make the whole price snapshot atomic. A reader can still see a mixed state, for example a new `bid` with an old `ask`, because atomicity here is per field, not across multiple fields updated as one logical unit.[105] [106]

So the remaining race is a consistency/publication race even though the raw 64-bit write is atomic. To solve that, I'd use a publication protocol around `seq`: writer updates payload fields,

then publishes the sequence; reader validates by checking sequence stability before and after reading the payload.[107] [108]

⁂

Correct. Per-field atomicity, not snapshot atomicity. That's the right distinction and you landed on the right solution direction — a sequence protocol.

Now show me it works.

Write the reader and writer methods for `PaddedPriceSlot` using the sequence protocol you just described. I want to see the actual Java — both sides. Then tell me what memory ordering guarantees make this correct, and whether `volatile` on `seq` alone is sufficient to enforce them.

I'd implement it like a lightweight seqlock with a single writer. The writer marks the slot unstable, writes the payload, then publishes a stable even sequence; the reader retries unless it sees the same even sequence before and after reading:

```
public final class PaddedPriceSlot {
    long p01, p02, p03, p04, p05, p06, p07;

    volatile long seq;
    double bid;
    double ask;

    long p11, p12, p13, p14, p15, p16, p17;

    public void write(double newBid, double newAsk) {
        long s = seq;
        seq = s + 1;      // odd => write in progress
        bid = newBid;
        ask = newAsk;
        seq = s + 2;      // even => fully published
    }

    public PriceSnapshot read() {
        for (;;) {
            long s1 = seq;
            if ((s1 & 1L) != 0L) {
                continue;
            }

            double b = bid;
            double a = ask;

            long s2 = seq;
            if (s1 == s2) {
                return new PriceSnapshot(b, a, s2);
            }
```

```
        }
    }
}
```

This is correct because a write to a volatile field has release semantics, and a read of that same volatile field has acquire semantics, creating a happens-before edge on `seq`. That means the ordinary writes to `bid` and `ask` that occur before the final volatile write to `seq` become visible to a reader that observes that published `seq` value and then reads the payload.[118] [119] [120] [121]

Yes, `volatile` on `seq` alone is sufficient for the ordering, provided there is a single writer and all publication/validation goes through that same volatile field. I would not keep `bid` and `ask` volatile in this pattern; that adds extra barriers without improving snapshot correctness.[122] [119] [123]

⁂

Good instinct on the seqlock pattern. The structure is recognizable and mostly correct.

But I see a problem. A critical one.

You removed `volatile` from `bid` and `ask`. You're relying on `volatile seq` to provide ordering guarantees for the non-volatile payload fields.

Does the Java Memory Model actually guarantee that? Specifically — does a volatile write to `seq` establish a happens-before relationship that ensures `bid` and `ask` writes are visible to the reader before `seq = s + 2` is observed? Walk me through the JMM reasoning precisely.

Yes — I'd rely on the JMM for that publication pattern, but the guarantee comes from the final `seq = s + 2` volatile write being observed by a reader of that same `seq` field.[133] [134]

In the writer, the plain writes to `bid` and `ask` occur before the final volatile write in program order, and a volatile write to `seq` synchronizes-with a subsequent volatile read of `seq` by another thread. In the reader, once `s1 = seq` performs that volatile read, its later reads of `bid` and `ask` are ordered after it by program order, so by transitivity those payload writes happen-before the reader's payload reads.[134] [133]

So yes, `volatile` on `seq` alone is sufficient for visibility and ordering of `bid` and `ask`; they do not also need to be volatile. The caveat is that correctness still depends on a single writer and the full double-check sequence protocol, otherwise I can still observe an inconsistent snapshot.[133] [134]

⁂

Close. But not quite complete — and in a system like IG's pricing engine, "close" is where bugs live.

You've correctly described the happens-before chain for visibility. But you've skipped over something important on the CPU side.

The JMM gives you a high-level guarantee, but the JIT compiler and the CPU are both allowed to reorder instructions as long as they don't violate the JMM contract **within a single thread**.

Here's the specific risk I want you to address:

In your writer:

```
seq = s + 1;      // volatile write
bid = newBid;      // plain write
ask = newAsk;     // plain write
seq = s + 2;      // volatile write
```

The JMM says a volatile write cannot be reordered with a preceding write. But what about the plain writes to `bid` and `ask` — can the JIT or CPU reorder them **after** the final `seq = s + 2` volatile write?

Answer precisely. If there is a risk, tell me what prevents it. If there is no risk, tell me exactly which JMM rule closes the door.

No — in this pattern, the JMM does **not** allow `bid` and `ask` stores that occur before the final volatile `seq` store in program order to be reordered to after it. The rule that closes the door is the volatile-store ordering rule: normal reads and normal writes that come before a volatile write cannot be moved past that volatile write, and a volatile write has the same memory effect as a monitor release.[135] [136]

So in the writer, `bid = newBid; ask = newAsk; seq = s + 2;` means the payload writes are ordered before the publishing volatile write. Once a reader observes that `seq` value via a volatile read, the synchronizes-with / happens-before chain makes those earlier payload writes visible to that reader.[137] [136] [135]

At the CPU level, HotSpot is free to choose the fence strategy, but it must emit code strong enough to preserve the JMM contract, typically by preventing StoreStore reordering around the volatile publication. So `volatile seq` alone is sufficient for the ordering here.[136] [137] [135]

⁂

Good. That's a solid and precise answer. You correctly cited StoreStore ordering and the release semantics of a volatile write. You're demonstrating real JMM depth, not just surface knowledge.

Let's shift pressure now.

You've built a clean single-writer seqlock. But we're at IG. We have **10,000 instruments**. Market data arrives from multiple venues — LSE, NASDAQ, dark pools — and different instruments are owned by different feed handler threads. So we no longer have a single writer globally. We have **multiple writers, each owning a partition of instruments**.

Here's the scenario:

Your pricing engine receives an update for `instrument 42`. That update needs to:

1. Update the `PaddedPriceSlot` for instrument 42
2. Notify 30 downstream execution threads that a new price is available so they can evaluate open orders against it

You've already solved the slot update. Now I want to focus on the **fan-out problem**.

How do you get that price update to 30 execution threads with minimum latency and no lock contention? Walk me through your design. Be specific about data structures, threading model, and the tradeoffs you're making.

I'd treat the slot update and the fan-out as two different concerns. The latest-price cache answers "what is the current price for instrument 42," while the notification path answers "which execution engines need to react to update 42 right now." [150] [151]

For fan-out, I would use a Disruptor-style **multicast** ring buffer, not 30 separate queues. The feed handler that owns instrument 42 publishes a tiny event like `{instrumentId, slotSeq}` into a preallocated ring, and each of the 30 execution threads has its own consumer sequence over the same ring so the event is delivered to all consumers without lock contention or per-consumer copying. [152] [151] [150]

Threading-wise, I'd keep partitioned feed handlers as producers and bind each execution thread to a fixed responsibility set, ideally with stable CPU affinity if the platform allows it, so each consumer can evaluate orders deterministically and with warm caches. If I have multiple producers, I'd use the Disruptor's multi-producer sequencer, knowing it introduces CAS on claim, but I still avoid the much worse cost of 30 queue writes per update. [151] [153]

The event itself should be minimal: instrument id, maybe venue or update type, and the published slot sequence so consumers can detect staleness and skip redundant work. I would not put full price objects on the ring, because that duplicates data and increases GC pressure; consumers can read the latest snapshot from the instrument slot after receiving the notification. [150] [151]

The main tradeoff is slow-consumer pressure. In a pure multicast ring, the producer cannot overrun the slowest gated consumer, so one lagging execution thread can throttle the system unless I isolate consumers into tiers, use dependency graphs, or allow some consumers to be lossy/coalescing when only the latest price matters. In practice, for pricing I'd often prefer "latest wins" semantics per instrument, so I'd combine the multicast notification ring with coalescing logic in consumers or shard downstream consumers by instrument partitions to keep the hot path bounded.[154] [155] [156] [150]

⁂

What exactly does the multi-producer sequencer do with CAS, why does it need it, and what is the concrete performance difference versus a single-producer sequencer? Then tell me — given our partitioned feed handler design, can we actually avoid the multi-producer sequencer entirely, and if so, how?

The multi-producer sequencer uses CAS to let several producer threads safely claim the next sequence numbers in the same ring without taking a lock. Each producer reads the current cursor, computes the next slot or batch, and uses CAS to atomically advance the shared claim sequence only if no other producer beat it to that range. It needs CAS because sequence reservation is shared mutable state; without an atomic claim step, two producers could publish into the same slot or corrupt ordering.[180] [181] [182]

The concrete performance cost versus single-producer is that the single-producer sequencer can use a plain monotonic counter with fewer coordination barriers, while multi-producer pays extra CAS retries, cache-line bouncing, and more contention on the shared cursor. The Disruptor user guide's own benchmark example shows single-producer throughput around roughly 77–89 million ops/sec versus about 26–30 million ops/sec in the compared multi-producer run, which illustrates the scale of the penalty even though exact numbers depend on workload and hardware.[181] [183]

Given our partitioned feed-handler design, yes, we can often avoid the multi-producer sequencer entirely. The trick is not to have all feed handlers publish into one shared ring; instead, give each partition or feed-handler thread its own single-producer ring, then route instruments so each ring has exactly one publishing thread.[184] [183]

From there I'd do one of two things. Either each execution thread consumes from multiple single-producer rings and merges by sequence or timestamp if ordering matters, or I insert a dedicated aggregator stage that drains the per-partition single-producer rings and republishes onto a downstream ring or directly triggers execution partitions. That trades some topology complexity for lower producer-side latency and much less CAS contention, which is usually the right trade in a hot market-data path.[183] [181]

⁂

Can we avoid the multi-producer sequencer entirely given our partitioned feed handler design? And if so, how — be specific.

Yes — we can avoid the multi-producer sequencer entirely if we keep the single-writer property **per ring**.[195] [196]

The concrete design is:

- Partition instruments so each feed-handler thread owns a disjoint subset.
- Give each feed-handler its own **single-producer ring buffer**.
- On an update, the feed handler writes the price slot and publishes a tiny notification event into **its own** ring.
- Each execution thread consumes from the set of rings it cares about, or from a downstream aggregation stage that is itself single-producer.[197] [196]

That works because the only reason the multi-producer sequencer exists is to coordinate several writers contending for the same cursor. If each ring has exactly one publisher, that contention disappears and the sequencer can stay single-producer.[196] [195]

The tradeoff is topology complexity rather than CAS contention. You may need one consumer loop that polls multiple rings, or a dispatcher/aggregator stage that merges per-partition events into per-execution-thread work queues, but the producer side stays simple and extremely fast.[198] [197]

For IG's pricing path, that is usually the right move: keep publishers partitioned, keep rings single-producer, and push any merge or routing cost to a later stage that is off the hottest path.[199] [196]

⁂

wait, so, we'd have multiple rings? one per instrument?

No — not one ring per instrument. That would be far too much overhead and complexity for 10,000 instruments.[212] [213]

The usual design is **one ring per partition or feed-handler thread**, where a partition owns a set of instruments. A feed handler updates the `PaddedPriceSlot` for the instrument, then publishes a small event into its partition ring; consumers downstream react based on the instrument id or partition id, not by having a separate ring per symbol.[214] [213]

So the granularity is:

- one writer per partition,
- one ring per writer,
- many instruments per partition,
- many consumers reading from those rings or from a later merged stage.[213] [215]

If you did one ring per instrument, you'd burn memory, bookkeeping, and scheduling overhead, and you'd also make routing and consumer management ugly. For IG-scale pricing, partitioning by venue, instrument family, or hash bucket is the practical move.[216] [214] [212] [213]

⁂

ok, so what is the partition here?

Not usually. The partition is the unit of ownership, not the individual instrument.[228] [229]

In this design, a partition could be:

- a venue feed, like LSE or NASDAQ.
- a hash bucket of instruments.
- a business domain slice, like UK equities vs. FX vs. futures.
- a shard chosen so one feed-handler thread owns that group of instruments end-to-end.[229] [230] [228]

The point is that every instrument belongs to exactly one partition at a time, so one feed-handler thread updates its slot and publishes into its own single-producer ring. That gives us single-writer semantics per partition without needing a ring per instrument or a multi-producer sequencer.[230] [231] [232] [229]

If the interviewer asked me to pick one for IG, I'd say: partition by the feed-handler ownership boundary first, then within that partition keep a stable instrument map and a single-producer notification path.[228] [229]

⁂

can you show a barebones component/flow diagram in mermaid?

Yes — here's a barebones Mermaid flow that matches the design:

```
flowchart LR
  A[Feed handler<br/>partition owner] --> B[Update PaddedPriceSlot<br/>for instrum
  B --> C[Publish event<br/>{instrumentId, slotSeq}<br/>to single-producer ring]
  C --> D[Exec thread 1<br/>consume event]
  C --> E[Exec thread 2<br/>consume event]
  C --> F[Exec thread 30<br/>consume event]
  D --> G[Read latest slot<br/>& evaluate orders]
```

```
  E --> G
  F --> G
```

I also saved the diagram text as an artifact .

syntax is incorrect

You're right — the earlier Mermaid snippet wasn't valid for every renderer.

Use this cleaner version:

```
flowchart LR
  A[Feed handler partition owner] --> B[Update slot for instrument 42]
  B --> C[Publish event to single-producer ring]
  C --> D[Exec thread 1]
  C --> E[Exec thread 2]
  C --> F[Exec thread 30]
  D --> G[Read latest slot and evaluate]
  E --> G
  F --> G
```

If you want line breaks inside nodes, some Mermaid renderers prefer `<br/>` or require different configuration, so I kept the labels single-line to stay portable.[245] [246]

⁂

ok, now a class diagram to solve this

Yes — here's a barebones Mermaid **class diagram** for the design we've been discussing:

```
classDiagram
    class FeedHandler {
        +int partitionId
        +onPriceUpdate(int instrumentId, double bid, double ask)
    }

    class PaddedPriceSlot {
        -volatile long seq
        -double bid
        -double ask
        +write(double bid, double ask)
        +read() PriceSnapshot
    }

    class PriceSnapshot {
        +double bid
        +double ask
        +long seq
    }

    class PartitionRingBuffer {
```

```
        +publish(PriceUpdateEvent event)
        +consume()
    }

    class PriceUpdateEvent {
        +int instrumentId
        +long slotSeq
    }

    class ExecutionThread {
        +onEvent(PriceUpdateEvent event)
        +evaluateOrders(int instrumentId)
    }

    class InstrumentRegistry {
        +slotFor(int instrumentId) PaddedPriceSlot
        +partitionFor(int instrumentId) int
    }

    FeedHandler --> InstrumentRegistry : resolves slot
    InstrumentRegistry --> PaddedPriceSlot : owns
    FeedHandler --> PaddedPriceSlot : writes
    FeedHandler --> PartitionRingBuffer : publishes to
    PartitionRingBuffer --> PriceUpdateEvent : contains
    ExecutionThread --> PartitionRingBuffer : consumes from
    ExecutionThread --> InstrumentRegistry : looks up slot
    PaddedPriceSlot --> PriceSnapshot : returns
```

If you want, I can also give you:

- a **cleaner interview-whiteboard version**,
- a **more realistic production version** with `Sequencer`, `EventProcessor`, and partition ownership,
- or a **sequence diagram** for one price update end-to-end.

walk me through each of these diagrams

Sure — here's the walkthrough, diagram by diagram, in the order we've been using them.

1. `FeedHandler`

This is the thread that owns a partition of instruments. Its job is to receive market data, map the instrument to the right slot, update that slot, and publish a lightweight notification event. In the Disruptor model, this is the producer side of the pipeline, and the key design goal is that it stays single-writer for its own partition.[260] [261]

2. `InstrumentRegistry`

This is the lookup layer that tells us where an instrument lives. For `instrumentId = 42`, it resolves the owning partition and the corresponding `PaddedPriceSlot`. In a real system, this would usually be a dense id-to-slot mapping rather than a general-purpose hash map, because this path is hot and needs to stay cache-friendly.[262] [261]

3. `PaddedPriceSlot`

This holds the latest price state for one instrument: `seq`, `bid`, and `ask`. The padding exists to keep adjacent slots from sharing cache lines and causing false sharing when different threads touch neighboring instruments. The `seq` field is the publication guard; the writer updates payload fields and then advances `seq`, while readers use `seq` to validate a coherent snapshot. [260] [261]

4. `PriceSnapshot`

This is just the immutable value the reader gets back after a successful sequence check. It avoids exposing live mutable state to downstream code. That keeps the execution thread logic simple: it either gets a valid snapshot or retries until the slot is stable.[263] [261]

5. `PartitionRingBuffer`

This is the notification channel for one partition or feed-handler owner. The producer publishes a tiny event like "instrument 42 updated at slot sequence 9001," and the ring fans that event to consumers without lock contention. In practice, this is where Disruptor-style sequencing and backpressure matter, because the ring is the coordination point between producer progress and consumer progress.[260] [261]

6. `PriceUpdateEvent`

This is deliberately small. It should carry just enough information for consumers to know what changed and where to read the current state, typically `instrumentId` and `slotSeq`. That keeps the ring buffer cheap and avoids copying full price objects into every downstream path. [264] [261]

7. `ExecutionThread`

This is one of the downstream consumers. It receives the notification, looks up the slot, reads the latest price, and evaluates open orders. In a Disruptor setup, each consumer has its own sequence so the system can track how far every consumer has progressed and avoid overwriting data too early.[260] [261]

8. `InstrumentRegistry -> PaddedPriceSlot`

That relationship means the registry owns the slot allocation and addressing. The practical meaning is: given an instrument id, we can jump directly to a stable memory location and read the latest price without recomputing structure or scanning. That's the hot-path optimization that makes the design scale.[262] [261]

9. `FeedHandler -> PaddedPriceSlot`

This is the write path. The feed handler is the only writer for that partition, so it can safely update the slot without a global lock. The publication sequence gives readers a way to detect when the slot contents are fully visible and stable.[260] [261]

10. `FeedHandler -> PartitionRingBuffer`

This is the handoff from data ingestion to fan-out. The feed handler publishes a small event after updating the slot, which wakes downstream execution threads without forcing them to poll a lock-protected cache. That keeps latency low and avoids contention on a shared monitor.[260] [261]

11. `PartitionRingBuffer -> PriceUpdateEvent`

This line means the ring carries the event objects. In a real low-latency design, those event objects are typically preallocated and reused, not created fresh on every tick, so the hot path stays GC-friendly.[260] [261]

12. `ExecutionThread -> PartitionRingBuffer`

This means the execution side consumes events from the ring. If we use a multicast pattern, each execution thread maintains its own sequence and sees each update according to the ring's dependency rules. That gives you fan-out without a separate queue per consumer.[260] [261]

13. `ExecutionThread -> InstrumentRegistry`

The execution thread uses the registry to find the current slot for the instrument mentioned in the event. It does not trust the event payload alone; it uses the event as a trigger and then reads the authoritative latest state from the slot. That prevents data duplication and keeps the update model consistent.[263] [261]

14. `PaddedPriceSlot -> PriceSnapshot`

This shows the read API boundary. The slot is mutable internally, but the returned snapshot is immutable and safe to pass around downstream. That keeps the execution logic clean and prevents accidental shared-state bugs.[260] [261]

If you want, I can turn this into a **sequence diagram** next so you can narrate the exact update flow from venue tick to order evaluation.

⁂

ADMIN

Hey, we're reaching the end of the time block. Time to wrap this up.

Understood — I'll wrap it up here.

1. https://www.iggroup.com/about-us/careers
2. https://www.jobsinforex.com/job/168913/technical-team-lead/
3. https://justjoin.it/job-offer/ig-group-lead-engineer-java-aws-typescript--krakow-java
4. https://talents.studysmarter.co.uk/companies/ig-group/london/java-engineer-core-trading-low-latency-33900585/
5. https://www.jobsinforex.com/company/7738/ig-group/
6. https://lseg.wd3.myworkdayjobs.com/en-US/Careers/job/Lead-Java-Developer--FX-Engineering_R0110451-1
7. https://bulldogjob.com/companies/jobs/234782-senior-java-engineer-sre-london-ig-group
8. https://www.fintechcareers.com/employer/ig-group/
9. https://lmax-exchange.github.io/disruptor/user-guide/index.html
10. https://lmax-exchange.github.io/disruptor/disruptor.html
11. https://www.baeldung.com/lmax-disruptor-concurrency
12. https://itnext.io/understanding-the-lmax-disruptor-caaaa2721496
13. https://blog.csdn.net/JackLoveBlueSky/article/details/8662418
14. https://blog.scottlogic.com/2021/12/01/disruptor.html
15. https://news.ycombinator.com/item?id=3173993
16. https://www.slideshare.net/slideshow/the-edge-2012-disruptor-guy-raz-nir-published/22790571
17. https://groups.google.com/g/lmax-disruptor/c/hb8BOkr0pjA
18. https://dev.to/kspeakman/explaining-the-lmax-disruptor-jkd
19. https://github.com/LMAX-Exchange/disruptor/blob/master/src/main/java/com/lmax/disruptor/RingBuffer.java
20. https://trishagee.com/2011/06/22/dissecting_the_disruptor_whats_so_special_about_a_ring_buffer/
21. https://stackoverflow.com/questions/64345270/how-is-lmax-disruptor-garbage-collector-friendly
22. https://betasignal.substack.com/p/from-wall-street-to-code-the-performance
23. https://martinfowler.com/articles/lmax.html
24. https://lmax-exchange.github.io/disruptor/disruptor.html
25. https://www.linkedin.com/posts/metta-naveen_java-concurrency-performance-activity-7420724166506524672-ReOx
26. https://lmax-exchange.github.io/disruptor/user-guide/index.html
27. https://www.linkedin.com/posts/augastine-ndeti-290230175_hft-lowlatency-garbagecollection-activity-7410907454995169280-pRNI

28. https://groups.google.com/g/lmax-disruptor/c/iaCod-YX9kY
29. https://www.youtube.com/watch?v=ResKWeoieR4
30. https://rtveliashvili.name/post/2023-04-29/gc-pauses-in-high-frequency-trading/
31. https://javadoc.io/doc/com.lmax/disruptor/3.0.0.beta2/com/lmax/disruptor/RingBuffer.html
32. https://stackoverflow.com/questions/18968981/high-frequent-concurrency-for-cache
33. https://github.com/LMAX-Exchange/disruptor/blob/master/src/main/java/com/lmax/disruptor/RingBuffer.java
34. https://dev.to/kspeakman/explaining-the-lmax-disruptor-jkd
35. https://www.baeldung.com/lmax-disruptor-concurrency
36. https://eng.wealthfront.com/2010/09/14/when-garbage-collection-is-failure-for/
37. https://lmax-exchange.github.io/disruptor/javadoc/com.lmax.disruptor/com/lmax/disruptor/RingBuffer.html
38. https://codesignal.com/learn/courses/advanced-real-life-concurrency-challenges/lessons/implementing-a-thread-safe-lru-cache-with-high-concurrency
39. https://www.linkedin.com/posts/narendrabareddy_java-multithreading-backenddevelopment-activity-7429737977242779648-GIIC
40. https://stackoverflow.com/questions/3951676/are-there-any-drawbacks-with-concurrenthashmap
41. https://stackoverflow.com/questions/59541893/performance-of-single-writer-different-reader-vs-same-thread
42. https://lmax-exchange.github.io/disruptor/disruptor.html
43. https://lmax-exchange.github.io/disruptor/user-guide/index.html
44. https://quant.engineering/ring-buffers-101.html
45. https://blog.scottlogic.com/2021/12/01/disruptor.html
46. https://dev.to/axelncho/using-lmax-disruptor-to-build-a-high-performance-in-memory-event-broker-in-java-6i
47. https://github.com/satyabratkumarsingh/lmax-disruptor-performance-test
48. https://github.com/janbodnar/Java-Advanced/blob/master/concurrent-collections.md
49. https://www.youtube.com/watch?v=ZjWh3Ou-H2k
50. https://www.linkedin.com/posts/harshamarnani_github-harsh160804disruptor-vs-blockingqueue-benchmark-activity-7424047116693471233-6qvG
51. https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/ConcurrentHashMap.html?spm=a2c6h.13046898.publish-article.72.34ec6ffaZNtvE2
52. https://news.ycombinator.com/item?id=40908273
53. https://www.reddit.com/r/java/comments/khl2n1/solution_for_hashmap_with_100m_values/
54. https://algomaster.io/learn/concurrency-interview/bounded-buffer
55. https://stackoverflow.com/questions/7216981/concurrenthashmap-implementation-and-limitations
56. https://www.baeldung.com/java-false-sharing-contended
57. https://jenkov.com/tutorials/java-concurrency/false-sharing.html

58. https://dev.to/ariasdiniz/understanding-and-solving-false-sharing-in-multi-threaded-applications-with-an-actual-issue-i-had-57c
59. https://sanjeev.pages.dev/false-sharing-cache-line-padding
60. https://sanjeev.pages.dev/false-sharing-cache-line-padding/
61. https://nimitsharma.wordpress.com/2025/06/14/deep-dive-java-object-layout-understanding-field-ordering-padding-and-alignment/
62. http://psy-lob-saw.blogspot.com/2014/06/notes-on-false-sharing.html
63. https://www.reddit.com/r/cpp/comments/13spubk/idiomatic_way_to_avoid_false_sharing/
64. https://www.linkedin.com/pulse/importance-structure-padding-computer-programming-sudhanshu-tripathi
65. https://blog.csdn.net/liuxiao723846/article/details/118423618
66. https://stackoverflow.com/questions/36258250/java-padding-performance-busting
67. https://en.wikipedia.org/wiki/False_sharing
68. https://stackoverflow.com/questions/77882130/how-minimize-false-sharing-in-this-array-processing-with-openmp
69. https://www.linkedin.com/pulse/minute-read-understanding-false-sharing-java-keshavram-kuduwa
70. https://javanexus.com/blog/eliminating-false-sharing-multithreading-performance
71. https://www.baeldung.com/java-false-sharing-contended
72. https://nimitsharma.wordpress.com/2025/06/14/deep-dive-java-object-layout-understanding-field-ordering-padding-and-alignment/
73. https://stackoverflow.com/questions/54493307/sun-misc-contended-is-not-a-member-of-package-sun-misc-scala/54493475
74. https://stackoverflow.com/questions/72674269/jdk-internal-vm-annotation-contended-didnt-work
75. https://docs.azul.com/prime/Contended
76. https://shipilev.net/talks/jvmls-July2013-contended.pdf
77. https://www.rapid7.com/blog/post/2016/03/14/a-point-of-contention-cache-coherence-on-the-jvm/
78. https://github.com/openjdk/jdk14u/blob/master/src/java.base/share/classes/jdk/internal/vm/annotation/Contended.java
79. https://kaustubhsaha.postach.io/post/false-sharing
80. http://robsjava.blogspot.com/2014/03/what-is-false-sharing.html
81. https://raw.githubusercontent.com/shipilev/jol/7fe001b81bebd4a8a68561d165fd1c4d45cbdd15/jol-samples/src/main/java/jdk/internal/vm/annotation/Contended.java
82. https://my.oschina.net/joeyjava/blog/4545431
83. https://docs.oracle.com/en/java/javase/11/migrate/index.html
84. https://www.javaspecialists.eu/archive/Issue249-Contended-since-9.html
85. http://www.aligelenler.com/2017/02/volatile-variable-false-sharing-and.html
86. https://juejin.cn/post/7255604228956110904

87. https://docs.oracle.com/javase/specs/jls/se8/html/jls-17.html
88. https://codexplo.wordpress.com/2015/12/07/longdouble-are-not-atomic-in-java/
89. https://stackoverflow.com/questions/34387242/why-using-volatile-makes-long-and-double-atomic
90. https://www.baeldung.com/java-volatile-vs-atomic
91. https://lmax-exchange.github.io/disruptor/disruptor.html
92. https://dzone.com/articles/longdouble-are-not-atomic-in-java
93. https://stackoverflow.com/questions/67935667/non-atomic-treatment-of-double-and-long
94. https://blog.csdn.net/aahyhaa/article/details/84412370
95. https://stackoverflow.com/questions/58308590/on-a-64-bit-jvm-is-reading-and-writing-of-a-double-atomic/58327589
96. https://stackoverflow.com/questions/517532/writing-long-and-double-is-not-atomic-in-java
97. https://betterprogramming.pub/a-deep-dive-into-the-java-volatile-keyword-7e1b9f9df604?gi=a78ed358cf41
98. https://www.reddit.com/r/java/comments/q7fj48/is_it_true_that_the_jvm_basically_stiches/
99. https://coderanch.com/t/574004/java/Long-double-atomic-java
100. https://marcos.github.io/java/threads/2019/09/16/64-bit-variables-and-atomicity-copy.html
101. https://www.oreilly.com/library/view/the-java-r-language/9780133260335/ch17lev1sec7.html
102. https://stackoverflow.com/questions/25070808/how-volatile-keyword-influences-atomicity-when-defining-a-long-or-double-variabl
103. https://docs.oracle.com/javase/specs/jls/se8/html/jls-17.html
104. https://www.oreilly.com/library/view/the-java-r-language/9780133260335/ch17lev1sec7.html
105. https://www.baeldung.com/java-volatile-vs-atomic
106. https://jenkov.com/tutorials/java-concurrency/java-happens-before-guarantee.html
107. https://stackoverflow.com/questions/17108541/happens-before-relationships-with-volatile-fields-and-synchronized-blocks-in-jav
108. https://www.javacodegeeks.com/2015/11/java-multi-threading-volatile-variables-happens-before-relationship-and-memory-consistency.html
109. https://stackoverflow.com/questions/34387242/why-using-volatile-makes-long-and-double-atomic
110. https://blog.csdn.net/weixin_31974757/article/details/114130042
111. https://bmwieczorek.wordpress.com/2015/10/21/java-memory-model/
112. https://www.geeksforgeeks.org/java/difference-between-atomic-volatile-and-synchronized-in-java/
113. https://dzone.com/articles/longdouble-are-not-atomic-in-java
114. https://www.hungrycoders.com/blog/understanding-thread-safety-in-java-atomic-volatile-and-synchronized
115. https://stackoverflow.com/questions/25070808/how-volatile-keyword-influences-atomicity-when-defining-a-long-or-double-variabl
116. https://courses.cs.taltech.ee/w/images/archive/a/ae/20170306132301!JavaFundamentals_-*Lecture_6*-_Threads_and_Java_Memory_Model.pdf

117. https://coderanch.com/t/574004/java/Long-double-atomic-java
118. https://jenkov.com/tutorials/java-concurrency/java-happens-before-guarantee.html
119. https://www.cs.umd.edu/~pugh/java/memoryModel/jsr-133-faq.html
120. https://www.geeksforgeeks.org/java/happens-before-relationship-in-java/
121. https://docs.oracle.com/javase/specs/jls/se8/html/jls-17.html
122. https://heappulse.com/memory-ordering-in-java-opaque-acquire-release-volatile/
123. https://stackoverflow.com/questions/28711229/do-i-need-to-make-both-variables-volatile-in-reader-writer-scenario
124. https://dev.to/denis_4354a4219c6482/java-memory-model-explained-27l1
125. https://codingtechroom.com/question/-java-volatile-synchronized-read-write-lock
126. https://stackoverflow.com/questions/16615140/is-volatile-read-happens-before-volatile-write
127. https://shipilev.net/blog/2016/close-encounters-of-jmm-kind/
128. https://www.tdcommons.org/cgi/viewcontent.cgi?article=5770&context=dpubs_series
129. https://stackoverflow.com/questions/4289837/mix-volatile-and-synchronized-as-a-read-write-lock
130. https://stackoverflow.com/questions/64412838/happens-before-guarantee-in-volatile
131. https://davekilian.com/acquire-release.html
132. https://www.reddit.com/r/rust/comments/sga1i0/is_there_a_concept_of_happensbefore_in_rust/
133. https://docs.oracle.com/javase/specs/jls/se8/html/jls-17.html
134. https://www.cs.umd.edu/~pugh/java/memoryModel/jsr-133-faq.html
135. https://www.cs.umd.edu/~pugh/java/memoryModel/jsr-133-faq.html
136. https://gee.cs.oswego.edu/dl/jmm/cookbook.html
137. https://docs.oracle.com/javase/specs/jls/se8/html/jls-17.html
138. https://stackoverflow.com/questions/48534643/do-i-have-the-correct-understanding-of-can-reorder-rules-from-the-jsr-133-cookbo
139. https://dev.to/tomerbendavid/java-memory-model-deep-dive-visibility-reordering-and-the-truth-about-volatile-58gd
140. https://stackoverflow.com/questions/54081251/happens-before-and-reordering-of-volatile
141. https://stackoverflow.com/questions/79906927/does-the-java-memory-model-permit-reordering-writes-to-a-volatile-variable-into
142. https://www.reddit.com/r/learnprogramming/comments/7rtd0s/does_volatile_in_java_prevent_compiler_from/
143. http://www.cs.umd.edu/~pugh/java/memoryModel/CommunityReview.pdf
144. https://stackoverflow.com/questions/51647337/java-volatile-reordering-prevention-scope
145. https://dev.to/denis_4354a4219c6482/java-memory-model-explained-27l1
146. https://betterprogramming.pub/a-deep-dive-into-the-java-volatile-keyword-7e1b9f9df604
147. https://ericlippert.com/2014/03/26/reordering-optimizations/

148. https://stackoverflow.com/questions/76050313/why-there-is-no-loadstore-barrier-before-volatile-write-in-java-jsr-133-orderin
149. https://heappulse.com/memory-ordering-in-java-opaque-acquire-release-volatile/
150. https://lmax-exchange.github.io/disruptor/user-guide/index.html
151. https://lmax-exchange.github.io/disruptor/disruptor.html
152. https://www.baeldung.com/lmax-disruptor-concurrency
153. https://blog.scottlogic.com/2021/12/01/disruptor.html
154. https://stackoverflow.com/questions/11606102/solution-to-slow-consumereventprocessor-issue-in-lmax-disruptor-pattern
155. https://lmax-exchange.github.io/disruptor/user-guide/
156. https://stackoverflow.com/questions/6559308/how-does-lmaxs-disruptor-pattern-work/24253277
157. https://groups.google.com/g/beast-users/c/P7OmgU_1qCo
158. https://arxiv.org/html/2512.23768v1
159. https://oneuptime.com/blog/post/2026-01-07-go-fan-out-fan-in/view
160. https://github.com/SourenaMOOSAVI/Low-Latency-Market-Data-Processor
161. https://dagster.io/glossary/fan-out
162. https://qconlondon.com/london-2017/system/files/presentation-slides/latency_sensitive_microservices_in_java_-_qcon_london_2017.pdf
163. https://www.operion.io/learn/component/fan-out-fan-in
164. https://dzone.com/articles/real-time-market-data-processing-designing-systems
165. https://axiom.co/blog/building-haydex
166. https://www.coudo.ai/blog/design-a-real-time-market-data-system
167. https://www.vldb.org/pvldb/vol18/p1522-fan.pdf
168. https://www.reddit.com/r/java/comments/1ctpebe/low_latency/
169. https://java-design-patterns.com/patterns/fanout-fanin/
170. https://github.com/LMAX-Exchange/disruptor/issues/237
171. https://www.scribd.com/document/817844610/LMAX-Disruptor-User-Guide
172. https://www.slideshare.net/slideshow/disruptor/59351202?nway-content_model=D
173. https://github.com/LMAX-Exchange/disruptor/issues/250
174. https://lmax-exchange.github.io/disruptor/files/Disruptor-1.0.pdf
175. https://javadoc.io/static/com.lmax/disruptor/3.3.2/com/lmax/disruptor/RingBuffer.html
176. https://trishagee.com/2011/07/10/dissecting_the_disruptor_wiring_up_the_dependencies/
177. https://cloud.tencent.com/developer/article/1696507
178. https://ipullrank.com/query-fan-out-personalization
179. https://www.janestreet.com/tech-talks/building-an-exchange/

180. https://lmax-exchange.github.io/disruptor/javadoc/com.lmax.disruptor/com/lmax/disruptor/MultiProducerSequencer.html
181. https://lmax-exchange.github.io/disruptor/disruptor.html
182. https://groups.google.com/g/lmax-disruptor/c/wNgGjTtbNxM/m/aIvQyRaUCQAJ
183. https://lmax-exchange.github.io/disruptor/user-guide/index.html
184. https://news.ycombinator.com/item?id=13102721
185. https://www.scribd.com/document/988441792/11-LMAX-Disruptor-Pattern-Ultra-Low-Latency-Ring-Buffer-Design-Executive-Summary-and-Takeaways
186. https://techishthoughts.com/posts/2026/01/off-heap-mpmc-queue-production/
187. https://www.facebook.com/groups/Mpcsoftware/posts/3010594862314045/
188. https://www.oreateai.com/blog/indepth-analysis-of-disruptor-principles-and-usage/2760080a72f7cfa070c1963174a3f6d9
189. https://github.com/LMAX-Exchange/disruptor/issues/250
190. https://www.youtube.com/watch?v=-6Q4jhhnnV4
191. https://stackoverflow.com/questions/2151780/is-multiple-producer-single-consumer-possible-in-a-lockfree-setting
192. https://www.reddit.com/r/programming/comments/2ll7bf/i_finally_finished_my_multiproducer_multiconsumer/
193. https://stackoverflow.com/questions/18997398/lmax-disruptor-is-too-slow-in-multi-producer-mode-compared-to-single-producer-mo
194. https://www.oreateai.com/blog/indepth-analysis-of-the-highperformance-lowlatency-messaging-framework-disruptor/33f8f0cc9084dd6fc4b4fb6eb9163f03
195. https://javadoc.io/static/com.lmax/disruptor/3.0.0/com/lmax/disruptor/MultiProducerSequencer.html
196. https://lmax-exchange.github.io/disruptor/user-guide/index.html
197. https://stevana.github.io/parallel_stream_processing_with_zero-copy_fan-out_and_sharding.html
198. https://news.ycombinator.com/item?id=13102721
199. https://lmax-exchange.github.io/disruptor/disruptor.html
200. https://beefed.ai/en/lock-free-ring-buffer-design-linux
201. https://groups.google.com/g/lmax-disruptor/c/FSeiAcQWs6o
202. https://groups.google.com/g/lmax-disruptor/c/SDTXfZnyguw
203. https://stackoverflow.com/questions/29820384/apache-kafka-order-of-messages-with-multiple-partitions
204. https://stackoverflow.com/questions/54359988/optimal-size-for-a-ring-buffer-with-single-producer-and-single-consumer
205. http://psy-lob-saw.blogspot.com/2014/04/notes-on-concurrent-ring-buffer-queue.html
206. https://inria.hal.science/hal-01446600/document
207. https://github.com/szanni/ringbuf
208. https://github.com/LMAX-Exchange/disruptor/issues/250

209. https://www.reddit.com/r/modular/comments/v84j3y/am_i_wrong_to_be_looking_for_an_alternative/
210. https://arxiv.org/pdf/2309.04918.pdf
211. https://www.renesas.com/en/video/single-ended-fanout-buffers-and-dividers-idt
212. https://martinfowler.com/articles/lmax.html
213. https://lmax-exchange.github.io/disruptor/user-guide/index.html
214. https://dzone.com/articles/real-time-market-data-processing-designing-systems
215. https://stevana.github.io/parallel_stream_processing_with_zero-copy_fan-out_and_sharding.html
216. https://docs.stacresearch.com/system/files/resource/files/STAC-Summit-4-May-2023-NovaSparks.pdf
217. https://gearspace.com/board/presonus-studio-one/1160461-studio-one-pro-3-5-kontakt-5-when-using-low-latency-engine.html
218. https://newsletter.systemdesign.one/p/disruptor-pattern
219. https://developer.nvidia.com/blog/optimizing-for-low-latency-communication-in-inference-workloads-with-jax-and-xla/
220. https://www.linkedin.com/posts/freederia_adaptive-bayesian-code-partitioning-for-lowlatency-activity-7434797999266066432-TP4T
221. https://bitsharestalk.org/index.php?topic=18314.0
222. https://www.reddit.com/r/musicproduction/comments/k17bvu/what_makes_low_latency_on_audio_interface/
223. https://groups.google.com/g/lmax-disruptor/c/KsROT6yOesA
224. https://www.quantbeckman.com/p/with-code-data-low-latency-data-structures
225. https://systemdr.substack.com/p/designing-for-low-latency-trading
226. https://arxiv.org/pdf/2309.04259.pdf
227. https://stackoverflow.com/questions/31966781/disruptor-ring-buffer
228. https://kx.com/products/kx-feedhandlers/
229. https://repository.tudelft.nl/file/File_a0937bef-756e-4d2a-9e72-c2cc40e75f41
230. https://webthesis.biblio.polito.it/14356/1/tesi.pdf
231. https://lmax-exchange.github.io/disruptor/user-guide/index.html
232. https://javadoc.io/static/com.lmax/disruptor/3.0.0/com/lmax/disruptor/MultiProducerSequencer.html
233. https://coincryptorank.com/blog/real-time-data-arb
234. https://ijetcsit.org/index.php/ijetcsit/article/download/100/77
235. https://tibco-computedb.readthedocs.io/en/docv1.1.0/best_practices/design_principles/
236. https://aws.amazon.com/blogs/database/speed-up-time-series-data-ingestion-by-partitioning-tables-on-amazon-rds-for-postgresql/
237. https://www.usenix.org/system/files/srecon23apac_slides-kasichayanula.pdf
238. https://www.forexvps.net/resources/low-latency-trading-infrastructure/
239. https://www.yugabyte.com/blog/how-to-achieve-high-availability-low-latency-gdpr-compliance-in-a-distributed-sql-database/

240. https://connect2.euronext.com/sites/default/files/it-documentation/SATURN Commodities Positions Reporting – Detailed Functional Specifications – Euronext Derivatives Markets - External - v5.354.0 + TC.pdf
241. https://www.actian.com/partitioning-how-to-do-it/
242. https://www.databricks.com/blog/architecting-high-concurrency-low-latency-data-warehouse-databricks-scales
243. http://acs.ict.ac.cn/asbd2015/202203/P020220329618935846776.pdf
244. https://www.coudo.ai/blog/design-a-real-time-market-data-system
245. https://mermaid.ai/open-source/syntax/flowchart.html
246. https://sidharthv96.github.io/mermaid/syntax/flowchart.html
247. https://docs.min87.com/en/mermaid/syntax/flowchart.html
248. https://mermaid.js.org/syntax/flowchart.html
249. https://docs.readme.com/main/docs/creating-mermaid-diagrams
250. https://mermaid.js.org/syntax/flowchart
251. https://github.com/mermaid-js/mermaid/issues/2428
252. https://mermaid.ai/open-source/syntax/examples.html
253. https://stackoverflow.com/questions/51366763/mermaid-diagram-line-break
254. https://github.com/langgenius/dify/issues/13481
255. https://mermaid.js.org/intro/syntax-reference.html
256. https://stackoverflow.com/questions/51366763/mermaid-diagram-line-break/51366989
257. https://www.scribd.com/document/482952814/Mermaid-pdf
258. https://emersonbottero.github.io/mermaid-docs/syntax/flowchart.html
259. https://github.com/mermaid-js/mermaid/blob/master/docs/syntax/flowchart.md
260. https://lmax-exchange.github.io/disruptor/user-guide/index.html
261. https://lmax-exchange.github.io/disruptor/disruptor.html
262. https://stevana.github.io/parallel_stream_processing_with_zero-copy_fan-out_and_sharding.html
263. https://groups.google.com/g/lmax-disruptor/c/Dqg9J6LFMx0
264. https://dev.to/axelncho/using-lmax-disruptor-to-build-a-high-performance-in-memory-event-broker-in-java-6i
265. https://gist.github.com/cornedriesprong/46589f2eaaf90b4b90346dfafcf956cb
266. https://www.baeldung.com/lmax-disruptor-concurrency
267. https://mermaid.js.org/syntax/architecture
268. https://stackoverflow.com/questions/13069970/multiple-producer-single-consumer-with-circular-buffer
269. https://github.com/LMAX-Exchange/disruptor/blob/master/src/main/java/com/lmax/disruptor/dsl/EventHandlerGroup.java
270. https://docs.mermaidviewer.com/diagrams/class.html
271. https://www.scribd.com/document/817844610/LMAX-Disruptor-User-Guide

272. https://lmax-exchange.github.io/disruptor/javadoc/com.lmax.disruptor/com/lmax/disruptor/dsl/Disruptor.html

273. https://pbs.bartificer.net/pbs141

274. https://www.iodraw.com/codechart/tutorial/en/classDiagram.html